

Guia de Desplegament

Agrupador de Càrregues

Versió 1.0 — Juny 2026

Plataforma objectiu: Ubuntu Server 24.04 LTS

Aplicació web Python/Flask amb PostgreSQL local i connexió a SQL Server

Contingut

1. Sobre l'aplicació
2. Requisits de la màquina virtual
3. Arquitectura de l'aplicació
4. Instal·lació de paquets
5. Configurar PostgreSQL
6. Descàrrega i configuració de l'aplicació
7. Verificació manual
8. Servei systemd amb Gunicorn
9. Apache VirtualHost (proxy invers)
10. Xarxa, DNS i tallafocs
11. Seguretat (auth, CSRF, audit, rate-limit)
12. Còpia de seguretat (PostgreSQL)
13. Llista de verificació
14. Manteniment i actualitzacions
15. Logs i monitoratge
16. Resolució de problemes
17. Resum i temps estimats

Nota: Aquesta aplicació comparteix servidor amb Lab FC, Comandes Venda i Visites. Molts dels paquets base (Python, Apache, ODBC Driver 18, PostgreSQL, UFW) ja estaran instal·lats. Les seccions afectades ho indiquen amb un avís.

Dependència obligatòria: L'app de **Comandes Venda** (`/var/www/comandes-venda`) **ha d'estar ja desplegada al mateix servidor abans** que aquesta. L'Agrupador importa el mòdul `motor.py` de Comandes Venda per calcular els embalatges (configurat via la variable `PREPARACIO_PATH`).

1. Sobre l'aplicació

Agrupador de Càrregues és una aplicació web interna que permet als operaris de l'oficina **agrupar càrregues** de venda i optimitzar la preparació de palets, reduint el temps d'operari al magatzem. Inclou:

- Llistat i cerca de càrregues amb filtres (transportista, data, estat, article).
- Vista **calendari mensual** de càrregues amb codis de color per transportista i indicador de granel.
- Agrupacions desades (identificadors UUID, plantilles, productes preparats al magatzem).
- API REST per al frontend (vanilla JS).

El sistema combina dues fonts de dades:

- **SQL Server** (remot, només lectura) — ERP `GWSV_AGRI` a `vkais:50058` . Conté les càrregues reals (taules `Cargas` , `Detcargas` , `ALBLINIA` , `ARTICLES` , etc.).
- **PostgreSQL** (local, lectura/escriptura) — persistència de les *agrupacions desades*, plantilles i productes marcats com a preparats.

2. Requisits de la màquina virtual

Maquinari

Recurs	Mínim	Recomanat
CPU	2 vCPU	4 vCPU
RAM	2 GB	4 GB
Disc	20 GB	40 GB
SO	Ubuntu Server 24.04 LTS	

Si Lab FC i Comandes Venda ja són al servidor, els recursos són suficients. Només cal afegir ~500 MB d'espai per aquesta app i ~50 MB/any per a la BD de PostgreSQL.

Programari necessari

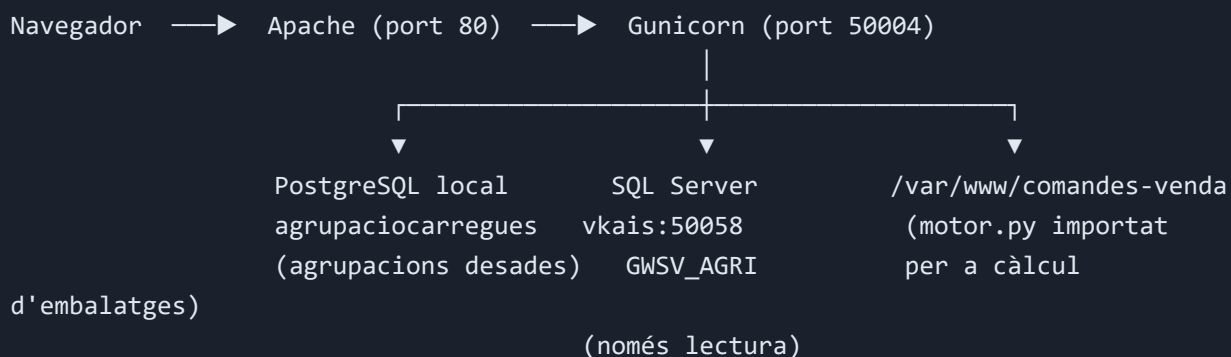
Component	Versió	Funció	Ja instal·lat amb Lab FC + Comandes Venda?
Ubuntu Server	24.04 LTS	Sistema operatiu	Sí
Python	3.10+	Runtime de l'aplicació	Sí
Apache2	2.4+	Proxy invers	Sí
ODBC Driver 18	18.x	Connexió a SQL Server	Sí (instal·lat amb Comandes Venda)
unixODBC	2.3+	Gestor ODBC a Linux	Sí (instal·lat amb Comandes Venda)
PostgreSQL	14+	Base de dades local	Sí (instal·lat amb Visites / Lab FC)
<code>libpq-dev</code>	—	Capçaleres per a <code>psycopg</code>	Sí (si hi ha Visites)
Git	2.x	Descarregar el codi	Sí

Connectivitat de xarxa

- Accés al servidor SQL Server `vkais` (port **50058**)
- Port 80 (Apache) per servir l'aplicació
- Port 50004 (intern, Gunicorn — només localhost)
- Port 5432 (PostgreSQL local — només localhost)
- Accés a Internet per descarregar paquets (durant la instal·lació)

3. Arquitectura de l'aplicació

L'aplicació segueix el mateix patró que les altres apps internes: Apache com a proxy invers davant de Gunicorn, que executa l'aplicació Flask. La nova característica respecte les anteriors és que combina **les dues fonts de dades**.



Estructura de directoris

```
/var/www/agrupacio-carregues/  
  app.py                # Servidor web Flask (punt d'entrada)  
  agregador.py          # Lògica d'agrupació per transportista  
  consultes_carregues.py # Consultes SQL Server (Cargas, ALBLINIA, ARTICLES...)  
  agrupacions_store.py  # CRUD PostgreSQL (agrupacions desades)  
  db.py                 # Pool de connexions a PostgreSQL  
  valida.py             # Validadors d'entrada (dates, codis...)  
  models_agrupacio.py   # Dataclasses del model  
  migrate_json_to_pg.py # One-shot migrador d'agrupacions antigues  
  requirements.txt       # Dependències Python  
  .env                  # Variables de configuració (cal crear manualment)  
  db/  
    schema.sql           # DDL: 3 taules + 1 vista  
    setup_local.sql       # Setup de dev (no s'usa en producció)  
  static/                # CSS, JavaScript  
  templates/             # HTML (Jinja2)  
  venv/                  # Entorn virtual Python
```

Comparativa amb les altres apps

Aspecte	Lab FC	Comandes Venda	Visites	Agrupador Càrregues
Directori	/var/www/lab-fc	/var/www/comandes-venda	/var/www/visites	/var/www/agrupacio-carregues
Servei	lab-fc.service	comandes-venda.service	visites.service	agrupacio-carregues.service
Port intern	8000	5001	50003	50004
ServerName	labfc.agrienergia.local	comandes.agrienergia.local	visitesfc.agrienergia.local	agrupacions.agrienergia.local
BD local	PostgreSQL	—	PostgreSQL	PostgreSQL
BD remota	—	SQL Server	—	SQL Server
Usuari SO	www-data	www-data	www-data	www-data

4. Instal·lació de paquets

4.1 Actualitzar el sistema

```
sudo apt update && sudo apt upgrade -y
```

4.2 Paquets base

Si Lab FC i Comandes Venda ja són al servidor, tots aquests paquets ja estaran instal·lats. Podeu saltar directament a la secció 5 (Configurar PostgreSQL).

```
sudo apt install -y python3 python3-pip python3-venv python3-dev \
    git curl gnupg2 apache2 \
    postgresql postgresql-contrib libpq-dev \
    build-essential
```

4.3 Driver ODBC 18 de Microsoft

Necessari per a connectar amb SQL Server. **Si Comandes Venda ja està desplegada, salta aquesta secció** — el driver ja hi és.

```
# Afegir el repositori de Microsoft
curl -fsSL https://packages.microsoft.com/keys/microsoft.asc \
    | sudo gpg --dearmor -o /usr/share/keyrings/microsoft-prod.gpg

# Per Ubuntu 24.04:
echo "deb [arch=amd64 signed-by=/usr/share/keyrings/microsoft-prod.gpg] \
    https://packages.microsoft.com/ubuntu/24.04/prod noble main" \
    | sudo tee /etc/apt/sources.list.d/mssql-release.list

# Instal·lar
sudo apt update
sudo ACCEPT_EULA=Y apt install -y msodbcsql18 unixodbc-dev
```

4.4 Activar mòduls d'Apache

```
sudo a2enmod proxy proxy_http headers rewrite expires
sudo systemctl restart apache2
```

4.5 Verificar instal·lacions

```
python3 --version      # 3.10+
psql --version          # 14+
apache2 -v              # 2.4+
git --version           # 2.x+
odbcinst -q -d          # Ha de mostrar: [ODBC Driver 18 for SQL Server]
```

5. Configurar PostgreSQL

L'aplicació utilitza una base de dades pròpia anomenada `agrupaciocarregues` dins del cluster PostgreSQL existent (el mateix que utilitza Visites o Lab FC). No interfereix amb cap altra base de dades del servidor.

5.1 Crear la base de dades i l'usuari

```
sudo -u postgres psql <<EOF
CREATE DATABASE agrupaciocarregues;
CREATE USER app_agrupacions WITH PASSWORD 'CANVIAR_CONTRASENYA_SEGURA';
GRANT CONNECT ON DATABASE agrupaciocarregues TO app_agrupacions;

\c agrupaciocarregues
ALTER DATABASE agrupaciocarregues OWNER TO app_agrupacions;
ALTER SCHEMA public OWNER TO app_agrupacions;
GRANT USAGE, CREATE ON SCHEMA public TO app_agrupacions;
EOF
```

Principi de mínim privilegi: `app_agrupacions` és propietari de l'esquema `public` perquè ha de crear les seves taules (`schema.sql` al pas 6.6). Després del setup, podeu opcionalment revocar `CREATE` i deixar només `SELECT/INSERT/UPDATE/DELETE` a les taules existents. Mai cal `SUPERUSER` ni accés a altres bases de dades del cluster.

IMPORTANT (PostgreSQL 15+): L'esquema `public` no és escriuible per defecte encara que es tinguin `ALL PRIVILEGES` sobre la base de dades. Cal canviar-ne el propietari (`ALTER SCHEMA public OWNER TO app_agrupacions`) i donar `CREATE` . Si no, la càrrega del schema fallarà amb `permission denied for schema public` .

IMPORTANT: Substituir `CANVIAR_CONTRASENYA_SEGURA` per una contrasenya real. Evitar caràcters que trenquen URLs (`@` , `:` , `/` , `?` , `#` , espais). Anotar la contrasenya per al `.env` .

5.2 Verificar accés

```
psql -U app_agrupacions -d agrupaciocarregues -h localhost -c 'SELECT version();'
# Hauria de mostrar la versió de PostgreSQL i sortir sense errors
```

L'esquema (taules `agrupacions` , `productes_preparats` , `agrupacio_carregues` i vista `v_agrupacions_estat`) es carrega després, al pas 6.5, un cop clonat el repositori.

6. Descàrrega i configuració de l'aplicació

6.1 Clonar el repositori

```
# Crear directori (mateix patró que les altres apps)
sudo mkdir -p /var/www/agrupacio-carregues
cd /var/www/agrupacio-carregues

# Clonar el repositori
sudo git clone https://github.com/ohijazo/Agrupador-de-Carregues.git .

# Assignar propietat a www-data
sudo chown -R www-data:www-data /var/www/agrupacio-carregues
```

6.2 Crear el fitxer .env

IMPORTANT: El fitxer `.env` NO està al repositori per seguretat. Cal crear-lo manualment amb les credencials de connexió a les bases de dades.

```
sudo nano /var/www/agrupacio-carregues/.env
```

Contingut complet:

```
# --- SQL Server (ERP, només lectura) ---
SQL_SERVER=vkais,50058
SQL_DATABASE=GWSV_AGRI
SQL_USER=<usuari_sql>
SQL_PASSWORD=<contrasenya_sql>

# --- PostgreSQL (persistència local) ---
PG_HOST=localhost
PG_PORT=5432
PG_DATABASE=agrupaciocarregues
PG_USER=app_agrupacions
PG_PASSWORD=<contrasenya_pg_pas_5.1>
PG_POOL_MIN=1
PG_POOL_MAX=5

# --- Ruta al motor d'embalatges (app germana Comandes Venda) ---
PREPARACIO_PATH=/var/www/comandes-venda

# --- Power BI (endpoint /api/pbi/carregues) ---
# Genera-la amb: python3 -c "import secrets; print(secrets.token_urlsafe(32))"
PBI_API_KEY=<clau_pbi>
```

```
# --- Autenticació (Phase D de seguretat) ---
AUTH_ENABLED=true
# Secret per a signar les cookies de sessió (OBLIGATORI a producció).
# Genera-la amb: python3 -c "import secrets; print(secrets.token_urlsafe(48))"
FLASK_SECRET_KEY=<secret_long_aleatori>

# --- Health endpoint (manté false en producció) ---
EXPOSE_HEALTH_DETAIL=false

# --- Opcionals: ajustaments de rendiment/limits ---
# Llindar (ms) per a registrar peticions lentes al log; default 500.
SLOW_REQUEST_MS=500
# Cap màxim de files retornades per /api/pbi/carregues; default 5000.
PBI_MAX_FILES=5000
```

Port SQL Server: El servidor SQL utilitza el port **50058** (no el port estàndard 1433). És imprescindible incloure el port a `SQL_SERVER` separat per coma.

PREPARACIO_PATH: L'app importa el mòdul `motor.py` de Comandes Venda per al càlcul d'embalatges. Aquest valor ha d'apuntar al directori d'instal·lació de Comandes Venda (`/var/www/comandes-venda`). Si Comandes Venda no està desplegada, l'endpoint `/api/agrupar` fallarà.

6.3 Protegir el fitxer .env

```
sudo chmod 640 /var/www/agrupacio-carregues/.env
sudo chown www-data:www-data /var/www/agrupacio-carregues/.env
```


6.4 Variables de configuració (referència)

Variable	Valor	Descripció
SQL_SERVER	vkais,50058	Servidor SQL Server amb port
SQL_DATABASE	GWSV_AGRI	Base de dades ERP
SQL_USER	(proporcionat)	Usuari SQL (només lectura)
SQL_PASSWORD	(proporcionat)	Contrasenya SQL
PG_HOST	localhost	Servidor PostgreSQL local
PG_PORT	5432	Port PostgreSQL
PG_DATABASE	agrupaciocarregues	Base de dades local
PG_USER	app_agrupacions	Usuari PostgreSQL
PG_PASSWORD	(creada al pas 5.1)	Contrasenya PostgreSQL
PG_POOL_MIN	1	Connexions mínimes al pool
PG_POOL_MAX	5	Connexions màximes al pool
PREPARACIO_PATH	/var/www/comandes-venda	Ruta a Comandes Venda (motor d'embalatges)
PBI_API_KEY	(generada)	Clau del header X-Api-Key per a Power BI
AUTH_ENABLED	true	Activa login obligatori a la web
FLASK_SECRET_KEY	(generada)	Signa cookies de sessió (obligatori en prod)
EXPOSE_HEALTH_DETAIL	false	/health retorna només booleans
SLOW_REQUEST_MS	500	Opcional. Llindar (ms) per logar peticions lentes
PBI_MAX_FILES	5000	Opcional. Cap màxim de files al PBI

Permisos SQL: L'usuari SQL només necessita permisos de **lectura (SELECT)** sobre les taules: `Cargas` , `De tcargas` , `ALBLINIA` , `ARTICLES` , `CPALBARA` , `SERIEALB` , `TRANS` (i d'altres que ja usa Comandes Venda). Si Comandes Venda ja té un usuari SQL configurat, es pot reutilitzar el mateix.

6.5 Crear entorn virtual i instal·lar dependències

```
cd /var/www/agrupacio-carregues
sudo -u www-data python3 -m venv venv
sudo -u www-data venv/bin/pip install --upgrade pip
sudo -u www-data venv/bin/pip install -r requirements.txt
sudo -u www-data venv/bin/pip install gunicorn
```

6.6 Carregar l'esquema PostgreSQL

```
cd /var/www/agrupacio-carregues
psql -U app_agrupacions -d agrupaciocarregues -h localhost -f db/schema.sql
# Demanarà la contrasenya de app_agrupacions.
# Crea: agrupacions, productes_preparats, agrupacio_carregues, v_agrupacions_estat,
#      audit_logs i usuaris.
```

6.7 Crear el primer usuari administrador (si AUTH_ENABLED=true)

Si activeu l'autenticació, cal crear el primer usuari administrador **abans** d'iniciar el servei (si no, ningú podrà entrar). El script demana usuari + contrasenya per stdin:

```
cd /var/www/agrupacio-carregues
sudo -u www-data venv/bin/python scripts/crear_admin.py
# Input:
#   Username: admin
#   Nom complet: Administrador
#   Contrasenya: ***** (mínim 8 caràcters)
#   Repeteix-la: *****
```

Rols disponibles (permisos diferenciats):

Rol	Accés
admin	Tot. A més pot gestionar usuaris des de <code>/admin/usuaris</code> (botó  al header).
oficina	Cerca de càrregues, agrupar, desar, eliminar, calendari, magatzem (lectura).
magatzem	Només <code>/magatzem</code> (checklist productes preparats). En entrar a <code>/</code> es redirigit a <code>/magatzem</code> .

L'admin pot crear comptes addicionals des de la web; no cal tornar a usar el CLI un cop el primer admin estigui creat.

6.8 Verificar connexions a les dues BDs

Important: els scripts comencen per `import app` per forçar la càrrega del `.env`. Si fas `import db` o `import consultes_carregues` directament sense importar abans `app`, les variables d'entorn no estan lligades i fallarà amb "PG_HOST no està configurat".

```
cd /var/www/agrupacio-carregues

# 1) Connexió a SQL Server
sudo -u www-data venv/bin/python -c "
import app
```

```

from consultes_carregues import connectar
conn = connectar()
conn.execute('SELECT 1').fetchone()
print('SQL Server OK')
conn.close()
"

# 2) Connexió a PostgreSQL
sudo -u www-data venv/bin/python -c "
import app
import db
ok, msg = db.health_ok()
print('PostgreSQL OK' if ok else 'PostgreSQL FAIL: ' + msg)
"

```

Si falla la connexió SQL Server: Verificar que el port **50058** està obert al firewall del servidor Windows on corre SQL Server. Provar amb: `telnet vkais 50058` .

7. Verificació manual de l'aplicació

Abans de configurar-la com a servei, convé provar que arrenca correctament des de la consola:

7.1 Arrencar manualment

```

cd /var/www/agrupacio-carregues
sudo -u www-data venv/bin/python -c "from app import app; app.run(host='127.0.0.1',
port=50004)"

```

7.2 Provar des d'un altre terminal

```

# Health check (conté estat de les dues BDs i del motor)
curl http://localhost:50004/health
# Hauria de retornar JSON com:
# {"db": {"ok": true, "msg": ""},
#  "pg": {"ok": true, "msg": ""},
#  "motor": {"ok": true, "msg": ""}}

# Pàgina principal
curl -I http://localhost:50004/
# Hauria de retornar HTTP/1.1 200 OK

# Aturar el python amb Ctrl+C

```

Si `health` retorna `motor.ok = false`, comprovar que `PREPARACIO_PATH` apunta a un directori on existeix `motor.py` i que Comandes Venda està ben desplegada.

8. Servei systemd amb Gunicorn

Configurem un servei systemd que executa Gunicorn com a servidor WSGI, igual que les altres apps.

8.1 Crear el fitxer de servei

```
sudo nano /etc/systemd/system/agrupacio-carregues.service
```

Contingut:

```
[Unit]
Description=Agrupador de Carregues
After=network.target postgresql.service
Requires=postgresql.service

[Service]
Type=simple
User=www-data
Group=www-data
WorkingDirectory=/var/www/agrupacio-carregues
EnvironmentFile=/var/www/agrupacio-carregues/.env
ExecStart=/var/www/agrupacio-carregues/venv/bin/gunicorn \
    --bind 127.0.0.1:50004 \
    --workers 2 \
    --timeout 120 \
    --access-logfile /var/www/agrupacio-carregues/access.log \
    --error-logfile /var/www/agrupacio-carregues/error.log \
    app:app
Restart=always
RestartSec=5

[Install]
WantedBy=multi-user.target
```

Nota: Gunicorn escolta a `127.0.0.1:50004` (només localhost). L'accés extern es fa a través d'Apache.

8.2 Activar i iniciar el servei

```
sudo systemctl daemon-reload
sudo systemctl enable agrupacio-carregues
```

```
sudo systemctl start agrupacio-carregues
sudo systemctl status agrupacio-carregues
# Hauria de dir: active (running)
```

8.3 Comandes útils del servei

Comanda	Acció
<code>sudo systemctl start agrupacio-carregues</code>	Iniciar
<code>sudo systemctl stop agrupacio-carregues</code>	Aturar
<code>sudo systemctl restart agrupacio-carregues</code>	Reiniciar
<code>sudo systemctl status agrupacio-carregues</code>	Veure estat
<code>journalctl -u agrupacio-carregues -f</code>	Veure logs en temps real

9. Apache VirtualHost (proxy invers)

9.1 Crear el VirtualHost

```
sudo nano /etc/apache2/sites-available/agrupacio-carregues.conf
```

Contingut:

```
<VirtualHost *:80>
    ServerName agrupacions.agrienergia.local

    # IMPORTANT: Permetre barres codificades (%2F) a les URLs
    # L'aplicació usa URLs com /api/agrupacions/<id>/producte
    # i carrega_id en format 2026/01/0002266
    AllowEncodedSlashes NoDecode

    # Headers de seguretat
    Header always set X-Frame-Options "DENY"
    Header always set X-Content-Type-Options "nosniff"
    Header always set Referrer-Policy "strict-origin"

    ProxyPreserveHost On
    ProxyPass          /static/ !
    ProxyPass          / http://127.0.0.1:50004/ nocanon
    ProxyPassReverse   / http://127.0.0.1:50004/

    # Servir fitxers estàtics directament
    Alias /static/ /var/www/agrupacio-carregues/static/
```

```
<Directory /var/www/agrupacio-carregues/static/>
    Require all granted
    Options -Indexes
    ExpiresActive On
    ExpiresDefault "access plus 7 days"
</Directory>

ErrorLog ${APACHE_LOG_DIR}/agrupacio-carregues-error.log
CustomLog ${APACHE_LOG_DIR}/agrupacio-carregues-access.log combined
</VirtualHost>
```

CRÍTIC: Les directives `AllowEncodedSlashes NoDecode` i `nocanon` són **imprescindibles**. Sense elles, Apache rebutja amb error 404 les peticions que conté `carrega_id` amb barres (format `EJE/SCA/CAR`, p. ex. `2026/01/0002266`).

9.2 Activar el site i recarregar

```
sudo a2ensite agrupacio-carregues.conf
sudo apache2ctl configtest
# Hauria de dir: Syntax OK
sudo systemctl reload apache2
```

10. Xarxa, DNS i tallafocs

10.1 Tallafocs (UFW)

Si UFW està actiu, verificar que els ports necessaris són oberts. Cap port nou per a aquesta app respecte les altres.

```
sudo ufw status
sudo ufw allow 'Apache' # Probablement ja obert per Lab FC
sudo ufw allow OpenSSH  # Si no està obert
```

No cal obrir el port 50004 al tallafocs — Unicorn escolta només a 127.0.0.1 i només Apache hi accedeix internament.

Sí cal: Assegurar que el port **50058** està obert al tallafocs del servidor Windows on corre SQL Server, per permetre connexions des del servidor Linux.

10.2 DNS intern

Crear entrada al servidor DNS corporatiu perquè els clients puguin accedir per nom:

Tipus: A
Nom: agrupacions.agrienergia.local
Valor: <IP-DEL-SERVIDOR> # mateix que les altres apps

11. Seguretat

L'app aplica diverses capes de seguretat. Resum:

Capa	Mecanisme	Variable d'entorn
Autenticació web	Usuaris locals a PostgreSQL (taula <code>usuaris</code>) amb sessions per cookie signada	<code>AUTH_ENABLED=true</code> + <code>F</code> <code>LASK_SECRET_KEY</code>
CSRF	Double-submit cookie automàtic per a <code>PATCH/POST/DELETE</code> a <code>/api/agrupacions/*</code> i <code>/api/admin/*</code>	(cap)
Rate-limit	30 req/min/IP a <code>/api/pbi/carregues</code> ; 5 intents/min/IP a <code>/login</code>	(cap)
Permisos per rol	Decoradors <code>@requires_rol</code> als endpoints: <code>admin</code> , <code>oficina</code> , <code>magatzem</code> tenen accés diferenciat	(cap, basat en taula <code>usuaris</code>)
Audit log	Taula <code>audit_logs</code> a PostgreSQL: registra login OK/fallit, logout, desar/eliminar agrupació, marcar/desmarcar producte	(cap)
API key PBI	Header <code>X-Api-Key</code> obligatori a <code>/api/pbi/*</code>	<code>PBI_API_KEY</code>
Headers HTTP	<code>X-Content-Type-Options</code> , <code>X-Frame-Options: DENY</code> , <code>Referrer-Policy</code> , <code>CSP script-src 'self'</code>	(cap)
Health endpoint	No exposa missatges d'error interns a producció	<code>EXPOSE_HEALTH_DETAIL=false</code>

11.1 Consultar el log d'auditoria

```
psql -U app_agrupacions -d agrupaciocarregues -h localhost <<EOF
-- Últimes 50 accions
SELECT ts, user_name, ip, accio, target FROM audit_logs ORDER BY ts DESC LIMIT 50;

-- Logins fallits de l'última setmana
SELECT ts, ip, target FROM audit_logs
WHERE accio = 'login_fallit' AND ts > NOW() - INTERVAL '7 days'
ORDER BY ts DESC;

-- Agrupacions eliminades per usuari
SELECT user_name, COUNT(*) FROM audit_logs
WHERE accio = 'agrupacio_eliminada' GROUP BY user_name;
EOF
```

11.2 Gestionar usuaris

Hi ha **dues vies** per gestionar usuaris:

Recomanat — UI web: un usuari amb rol `admin` pot anar a `http://agrupacions.agrienergia.local/admin/usuaris` (botó  al header). Permet crear, editar (nom, rol, actiu), canviar contrasenya i desactivar tots els usuaris des del navegador, sense haver d'entrar al servidor.

L'altra via és la **CLI** (útil per a crear el primer admin o per a scripts d'automatització):

```
# Crear un usuari nou (interactiu)
sudo -u www-data venv/bin/python scripts/crear_admin.py \
    --username operario1 --nom "Operari 1" --rol oficina

# Desactivar un usuari (sense esborrar – manté la traçabilitat)
psql -U app_agrupacions -d agrupaciocarregues -h localhost \
    -c "UPDATE usuaris SET actiu = FALSE WHERE username = 'operario1';"

# Canviar contrasenya d'un usuari (genera el hash amb Python):
sudo -u www-data venv/bin/python -c "
import os
os.environ.setdefault('PG_HOST', 'localhost')
import auth, db
db.execute(
    'UPDATE usuaris SET password_hash = %s WHERE username = %s',
    (auth.hash_password('nova_contrasenya'), 'operario1')
)
print('OK')
"
```

12. Còpia de seguretat (PostgreSQL)

Les agrupacions desades viuen a la BD local `agrupaciocarregues`. Recomanem un backup diari amb retenció de 30 dies, igual que la resta d'apps amb PostgreSQL:

```
sudo mkdir -p /opt/backups/agrupacio-carregues
sudo chown postgres:postgres /opt/backups/agrupacio-carregues

sudo nano /etc/cron.d/agrupacio-carregues-backup
```

Contingut (els `\%` són necessaris perquè cron interpreta `%` com a salts de línia):

```
BACKUP_DIR=/opt/backups/agrupacio-carregues

# Backup diari a les 02:30
```



```
30 2 * * * postgres pg_dump agrupaciocarregues | \
    gzip > $BACKUP_DIR/agrupaciocarregues_$(date +%Y%m%d).sql.gz

# Eliminar backups > 30 dies (a les 03:30)
30 3 * * * root find $BACKUP_DIR -name "*.sql.gz" -mtime +30 -delete
```

La BD de SQL Server (ERP) no la gestiona aquesta app — el seu backup és responsabilitat de l'equip de SQL Server / ERP, no d'aquest desplegament.

13. Llista de verificació

Executar totes les comprovacions abans de donar per finalitzat el desplegament:

#	Verificació	Comanda	Resultat esperat
1	Driver ODBC instal·lat	<code>odbcinst -q -d</code>	[ODBC Driver 18 for SQL Server]
2	PostgreSQL actiu	<code>systemctl is-active postgresql</code>	active
3	Accés a la BD local	<code>psql -U app_agrupacions -d agrupaciocarregues -h localhost -c 'SELECT 1'</code>	1
4	Repositori clonat	<code>ls /var/www/agrupacio-carregues/app.py</code>	Fitxer existent
5	Fitxer .env creat	<code>sudo cat /var/www/agrupacio-carregues/.env</code>	Variables PG_*, SQL_*, PREPARACIO_PATH
6	Entorn virtual creat	<code>ls /var/www/agrupacio-carregues/venv/bin/python</code>	Fitxer existent
7	Schema PG carregat	<code>psql -U app_agrupacions -d agrupaciocarregues -c '\dt'</code>	Llista de 3 taules
8	Servei actiu	<code>systemctl status agrupacio-carregues</code>	active (running)
9	Port 50004 escoltant	<code>ss -tlnp grep 50004</code>	LISTEN 127.0.0.1:50004
10	Health check OK	<code>curl http://localhost:50004/health</code>	JSON amb db, pg, motor tots ok:true
11	Apache respon (vhost correcte)	<code>curl -I -H 'Host: agrupacions.agrienergia.local' http://localhost/</code>	HTTP/1.1 200 OK
12	Pàgina principal carrega	Obrir <code>http://agrupacions.agrienergia.local</code>	Llistat de càrregues visible
13	Calendari funciona	Obrir <code>http://agrupacions.agrienergia.local/calendari</code>	Calendari mensual visible
14	Agrupació (motor) funciona	Seleccionar càrregues i clicar "Agrupar"	Resultat amb embalatges

Verificació ràpida des del terminal

```
# 1. Servei actiu?
systemctl is-active agrupacio-carregues

# 2. Port obert?
ss -tlnp | grep 50004

# 3. Resposta HTTP?
curl -s -o /dev/null -w '%{http_code}' http://localhost:50004/
# Ha de retornar: 200
```

```
# 4. Health check?
curl -s http://localhost:50004/health | python3 -m json.tool
# Ha de retornar JSON amb db, pg i motor tots ok:true

# 5. SQL Server accessible?
telnet vkais 50058
# Ha de connectar (no "Connection refused")
```

14. Manteniment i actualitzacions

13.1 Actualitzar l'aplicació

Quan hi hagi una nova versió al repositori GitHub:

```
cd /var/www/agrupacio-carregues

# 1. Descarregar canvis
sudo -u www-data git pull origin main

# 2. Actualitzar dependències (si requirements.txt ha canviat)
sudo -u www-data venv/bin/pip install -r requirements.txt

# 3. Aplicar canvis al schema (si db/schema.sql ha canviat)
# El schema és idempotent (CREATE IF NOT EXISTS), s'hi pot tornar a passar.
psql -U app_agrupacions -d agrupaciocarregues -h localhost -f db/schema.sql

# 4. Reiniciar el servei
sudo systemctl restart agrupacio-carregues

# 5. Verificar
sudo systemctl status agrupacio-carregues
curl http://localhost:50004/health
```

El fitxer `.env` no es veu afectat per les actualitzacions (no està al repositori).

13.2 Rotació de logs

```
sudo nano /etc/logrotate.d/agrupacio-carregues
```

Contingut:

```
/var/www/agrupacio-carregues/*.log {
    daily
```

```
missingok
rotate 14
compress
delaycompress
notifempty
copytruncate
}
```

15. Logs i monitoratge

Fitxer / Comanda	Contingut
<code>/var/www/agrupacio-carregues/agrupacio.log</code>	Log de l'app (resultats, errors d'agrupació)
<code>/var/www/agrupacio-carregues/access.log</code>	Log d'accés HTTP (Gunicorn)
<code>/var/www/agrupacio-carregues/error.log</code>	Errors del servidor web (Gunicorn)
<code>/var/log/apache2/agrupacio-carregues-*.log</code>	Logs d'Apache
<code>journalctl -u agrupacio-carregues</code>	Logs del servei systemd
<code>/var/log/postgresql/</code>	Logs de PostgreSQL

```
# Logs de l'app en temps real
tail -f /var/www/agrupacio-carregues/agrupacio.log

# Logs del servei
journalctl -u agrupacio-carregues -f --no-pager

# Logs d'Apache
tail -f /var/log/apache2/agrupacio-carregues-error.log
```

16. Resolució de problemes

Problema	Causa probable	Solució
El servei no arrenca	Dependències o <code>.env</code> absent	<code>journalctl -u agrupacio-carregues -n 50</code>
<code>permission denied for schema public</code>	PostgreSQL 15+ amb permisos de <code>public</code> per defecte	Repetir <code>ALTER SCHEMA public OWNER TO app_agrupacions; GRANT CREATE ON SCHEMA public TO app_agrupacions;</code>
<code>Can't open lib ODBC 18</code>	Driver no instal·lat	Repetir pas 4.3
<code>Login timeout expired</code>	Port SQL bloquejat pel firewall	<code>telnet vkais 50058</code> — obrir port al firewall de Windows
<code>Login failed for user</code>	Credencials SQL incorrectes	Revisar <code>.env</code> (contactar responsable)
<code>health.motor.ok = false</code>	<code>PREPARACIO_PATH</code> incorrecte o Comandes Venda no desplegada	Verificar que <code>/var/www/comandes-venda/motor.py</code> existeix
<code>health.pg.ok = false</code>	Postgres no arrenca o credencials/host malament	<code>systemctl status postgresql</code> i revisar <code>PG_*</code> al <code>.env</code>
Error 404 al fer clic a una agrupació	Apache rebutja barres codificades	Afegir <code>AllowEncodedSlashes NoDecode</code> i <code>nocanon</code> (pas 9.1)
Error 502 Bad Gateway	Gunicorn no respon	<code>sudo systemctl restart agrupacio-carregues</code>
Pàgina no accessible	DNS o Apache	<code>curl http://localhost:50004/</code> + revisar entrada DNS
Calendari sense esdeveniments	Connexió a SQL Server caiguda	Mirar <code>health.db.msg</code> i fer <code>telnet vkais 50058</code>

17. Resum i temps estimats

Temps estimats assumint que Lab FC, Comandes Venda i Visites ja són al servidor (Python, Apache, ODBC 18, PostgreSQL, UFW ja configurats):

Pas	Descripció	Temps estimat
4	Verificar paquets base (cap nou)	2 min
5	Crear BD i usuari PostgreSQL	5 min
6	Clonar repo, crear <code>.env</code> i entorn virtual	10 min
6.6-6.7	Carregar schema i verificar connexions	5 min
7	Verificació manual de l'app	5 min
8	Servei systemd	5 min
9	Apache VirtualHost	5 min
10	DNS (entrada nova)	5 min
11	Backup PostgreSQL	5 min
12	Verificació final	5 min
	TOTAL ESTIMAT	50-60 min

Si Comandes Venda **encara no** està desplegada al servidor, cal fer-ho primer (afegeix uns 30-40 min addicionals i instal·la el driver ODBC 18 i les dependències SQL Server).

Comparativa de serveis al servidor (un cop desplegat)

	Lab FC	Comandes Venda	Visites	Agrupador Càrregues
Directori	<code>/var/www/lab-fc</code>	<code>/var/www/comandes-venda</code>	<code>/var/www/visites</code>	<code>/var/www/agrupacio-carregues</code>
Servei	<code>lab-fc.service</code>	<code>comandes-venda.service</code>	<code>visites.service</code>	<code>agrupacio-carregues.service</code>
Port intern	8000	5001	50003	50004
ServerName	<code>labfc.agrienergia.local</code>	<code>comandes.agrienergia.local</code>	<code>visitesfc.agrienergia.local</code>	<code>agrupacions.agrienergia.local</code>
BD local	PostgreSQL	—	PostgreSQL	PostgreSQL <code>agrupaciocarregues</code>
BD remota	—	SQL Server (port 50058)	—	SQL Server (port 50058)
Usuari SO	www-data	www-data	www-data	www-data